

React.js Project

Web Application Development Project for the Faculty of Computers and Information using React.js

1. Project Description

The task is to develop a modern, responsive web application representing the official website of the Faculty of Computers and Information. The website must provide a professional and user-friendly experience and present academic and administrative information, news, announcements, and college services in a clear and organized manner.

The project must be implemented using React.js while following modern frontend development practices, reusable component design, and a well-organized project structure.

2. Project Objectives

The objective is to build a complete web interface for the Faculty of Computers and Information that is easy to use for students, faculty members, and visitors works effectively on desktop, tablet, and mobile devices is fast, organized, and scalable uses React.js properly relies on reusable components and provides a professional design suitable for an official educational institution.

3. Required Technologies

The project must use React.js, JavaScript or TypeScript, HTML5, CSS3, responsive design, React Router for navigation, and Git/GitHub for version control.

The use of Vite, CSS Modules or Tailwind CSS, an appropriate icon library such as Lucide React or React Icons, and a state-management library only when actually needed is recommended.

It is not allowed to build the website as plain HTML/CSS without real React components, duplicate UI instead of creating reusable components, place the entire application in one file, overuse inline styles, or submit an unchanged ready-made template.

4. Required Pages

The website must include at least the following pages: Home, About, Departments, Programs, News, News Details, Announcements, Faculty, Services, Events, and Contact.

The Home page must include a Header/Navbar, college logo and name, Hero Section, introduction, news, announcements, programs, statistics, services, quick links, events, and a Footer. Departments, programs, news, faculty, and events must provide appropriate details, with dedicated detail pages wherever required.

5. Navigation

A clear and well-organized navigation system must include at least: Home, About, Departments, Programs, News, Announcements, Faculty, Services, Events, and Contact. On mobile devices, the Navbar must transform into an appropriate mobile menu.

6. UI/UX Requirements متطلبات التصميم

The design must be modern, professional, academic, clean, and easy to use. It should use a consistent color system, clear typography, balanced spacing, consistent cards and buttons, subtle professional hover effects and transitions, and no excessive visual effects. The overall visual identity must fit an official university faculty website.

7. Responsive Design

The website must work correctly on desktop, laptop, tablet, and mobile devices and should be tested at least at 1920px, 1440px, 1024px, 768px, 390px, and 360px. Unnecessary horizontal scrolling, off-screen elements, clipped text, distorted images, or a broken mobile navigation experience are not acceptable.

8. Language and Internationalization Using i18n

The website must be fully bilingual in Arabic and English, using real i18n internationalization for translation management and language switching. The preferred implementation is react-i18next with i18next, or another professional equivalent.

A clear Language Switcher must allow users to switch between Arabic and English from any page. All UI elements must be translated, including navigation items, buttons, headings, messages, forms, loading states, error states, and empty states.

Static text must not be hard-coded inside components in a way that prevents translation. Translation files must be organized, for example under locales/ar and locales/en, with clear and structured translation keys such as common.nav.home and news.title.

When Arabic is selected, RTL must be enabled and Arabic paragraphs must be right-aligned. When English is selected, LTR must be enabled and English paragraphs must be left-aligned. The selected language must remain active while navigating between pages, and all primary content must be available in both languages.

The two languages must not be implemented by duplicating every page. Validation, success, and error messages must also be translated. Switching languages must not break the design or responsive layout. The selected language should preferably be stored in localStorage or an equivalent mechanism so that it can be restored when the application is reopened. For bilingual documents and interfaces, any paragraph beginning with Arabic text must use Right-to-Left direction and right alignment, while any paragraph beginning with English text must use Left-to-Right direction and left alignment.

9. Data Management

Data must not be hard-coded randomly inside components. Organized data files should be created, such as news.js, departments.js, faculty.js, events.js, and services.js, and the UI should be generated from these data structures using map().

Content that requires translation must be connected to i18n translation keys or a bilingual data structure rather than duplicating translated content inside pages.

10. Components

The application must be divided into reusable components such as Navbar, Footer, Hero, SectionTitle, NewsCard, AnnouncementCard, FacultyCard, DepartmentCard, ProgramCard, EventCard, ServiceCard, SearchBar, Filter, Button, Modal, and Loading. Large components containing unrelated responsibilities should be avoided.

11. React Requirements

React must be used properly through components, props, state, conditional rendering, lists, events, forms, React Router, and hooks where appropriate. useState and useEffect should only be used when they serve an actual requirement, not merely for demonstration. Custom hooks are encouraged when they improve project organization.

12. Routing

React Router must be used with separate routes for the required pages, for example: /, /about, /departments, /departments/:id, /programs, /news, /news/:id, /announcements, /faculty, /faculty/:id, /services, /events, and /contact.

A dedicated 404 Not Found page must be provided for unknown routes.

13. Search & Filtering

Real search functionality must be implemented for news, faculty members, and announcements. Appropriate filtering should be available by department, category, date, or content type depending on the page.

14. Forms & Validation

The contact form must validate the name, email, phone number, and message fields, display clear validation errors, and show a success message after submission. All validation messages must be translated through i18n.

15. Loading & Empty States

The application must handle Loading, Empty, and Error states, such as “Loading data”, “No news available”, and “An error occurred while loading the data”, with an option to retry where appropriate. These messages must be available in both Arabic and English.

16. Images and Data

Mock data may be used, but it must be realistic and relevant to a Faculty of Computers and Information. Lorem ipsum must not be used as the main content. Images should be appropriate for educational and university purposes.

17. Accessibility

The project must follow accessibility basics, including semantic HTML, meaningful image alt text, labels for form fields, keyboard usability, visible focus states, sufficient contrast, and avoiding color as the only way to communicate information.

18. Performance

Performance must be considered by avoiding unnecessarily large images, using lazy loading where appropriate, preventing unnecessary re-renders, structuring components properly, and avoiding unnecessary libraries.

19. Project Structure

The project should follow a clear structure such as:

src/

└─ assets/

└─ components/

└─ pages/

|— layouts/

|— data/

|— hooks/

|— utils/

|— routes/

|— locales/

| |— ar/

| |— en/

|— App.jsx

|— main.jsx

|— index.css

Each folder must have a clear responsibility.

20. Git & GitHub

The project must be hosted on GitHub. The repository must contain a clear README, project description, setup instructions, technologies used, screenshots, and build/run instructions. Organized commits are recommended, for example: feat: add homepage, feat: add news section, feat: add faculty page, feat: add responsive navbar, fix: mobile layout issue, style: improve typography, and refactor: organize components.

21. README

The README must include a Project Overview, Features, Technologies, Installation, Build instructions, Screenshots, and clear steps for running the project locally.

22. Quality Standards

The project must be maintainable, scalable, and easy to extend. Components should be reusable, the code should be free from major errors, and the application should be responsive, fast, and readable.

23. Deliverables

1. Complete source code.
2. GitHub repository.
3. README.
4. Screenshots.
5. Production build.
6. Live demo if available.
7. Complete Arabic and English translation files.
8. Brief documentation explaining the i18n implementation and RTL/LTR switching.

24. Project Rules

1. React.js is mandatory.
2. Reusable components must be used.
3. React Router must be used.
4. The website must be responsive.
5. RTL and LTR must work according to the selected language.
6. i18n must be used to manage Arabic and English.
7. Unnecessary code duplication is not allowed.
8. The entire project must not be placed inside one component.
9. All required pages must be navigable.

10. Broken links are not acceptable.
11. There must be no obvious console errors.
12. Loading, empty, and error states must be handled.
13. Data must be organized.
14. Code must be readable and maintainable.
15. A consistent visual system must be maintained across the application.

25. Bonus Features

Bonus points may be awarded for features such as Dark Mode, a real Backend API, Authentication, an Admin Dashboard, news and announcement management, Pagination, Advanced Search, Notifications, PWA, SEO Optimization, professional animations, Unit Tests, and End-to-End Tests, provided that these features do not reduce the quality of the core requirements.

26. Evaluation Criteria

Total score: 100 points.

React & Code Quality — 25 points: proper React usage 7, component architecture 6, props/state/hooks 5, file organization 4, reusability 3. UI/UX — 20 points: design quality 6, color and typography consistency 4, usability 4, layout quality 3, interaction quality 3.

Responsive Design — 15 points: desktop 3, tablet 3, mobile 5, responsive navbar 2, no layout issues 2. Functionality — 15 points: routing 3, search 3, filtering 3, forms and validation 3, loading/error/empty states 3.

Project Architecture — 10 points: folder structure 3, data organization 2, reusable components 3, clean code 2.

Accessibility & Performance — 5 points: accessibility 2, image optimization 1, performance 2.

GitHub & Documentation — 5 points: README 2, Git history 1, project documentation 1, live demo 1.

Creativity & Extra Features — 5 points.

Correct i18n implementation and complete Arabic/English support are core requirements and must be evaluated within React, functionality, and architecture. They are not optional bonus features.

27. Penalties

Penalties may be applied as follows:

- 10 points for failing to use React properly or using React only superficially around plain HTML/CSS.
- 5 points for clearly broken responsive behavior.
- 5 points if most pages are incomplete or only placeholders.
- 5 points for obvious JavaScript errors in the console.
- 5 points for poor architecture with an excessively large main component.
- 3 points for broken pages or links.
- 3 points for excessive avoidable code duplication.
- 2 points for missing or inadequate README documentation.
- Up to 5 points for failing to implement real i18n, incomplete bilingual translation, or incorrect RTL/LTR behavior.

28. Project Acceptance Criteria

The project is considered successful when the following minimum requirements are met: all core pages work, React is used properly, routing works, the website is responsive, Arabic and English are fully supported through i18n, RTL/LTR behavior is correct for each language, search and filtering work, forms include validation, there are no major console errors, the project is well organized, the code is readable and maintainable, and the project can be run using the README instructions.

29. Final Evaluation Principles

The project will be evaluated based on implementation quality, not merely the number of pages. The evaluation focuses on Code Quality, UI/UX, React Architecture, Responsiveness, Functionality, Maintainability, and i18n. A large number of pages with poor design or unorganized code will not result in a high score. A well-structured project that uses React properly, provides an excellent user experience, and delivers real Arabic/English support can receive a high score even if some optional features are not implemented.

30. Final Requirement

Build a complete and modern Faculty of Computers and Information website using React.js. The result should look and behave like a real official university faculty website rather than a simple demo or a collection of disconnected pages.

The project must be Professional, Responsive, Accessible, Maintainable, Scalable, User Friendly, and fully Bilingual (Arabic/English), with complete compliance with all requirements, rules, and evaluation criteria described above.

One final note: We may not be fully familiar with every concept required in the project, such as **i18n**. However, the goal of this project is not only to apply what we have already learned, but also to use our existing knowledge and skills to **learn, understand, and implement new concepts independently**.

Thank you, and best of luck.